

Analiză Comparativă: Scala și Racket

Implementarea unui Calculator Funcțional

Radoi Vlad, Mincu Alin, Noje Vlad, Jugaru Paul

Tema Proiectului

Construirea unui **Calculator Funcțional** care poate să citească și să calculeze expresii matematice.

- Am ales două limbaje complet diferite pentru a vedea cum abordează fiecare aceeași problemă:
 - **Scala:** Un limbaj strict, care verifică totul înainte de rulare și se integrează cu Java.
 - **Racket:** Un limbaj flexibil, bazat pe liste, unde codul are aceeași formă cu datele.
- Obiectiv: Să comparăm cum definim, citim și calculăm structurile matematice (AST - Abstract Syntax Tree).

1. Scala: Ce este și cum funcționează?

- **Verificare Strictă:** Compilatorul găsește greșelile de tip înainte ca programul să ruleze.
- **Stil Hibrid:** Ne permite să folosim atât clase (ca în Java), cât și funcții matematice simple.
- **Interoperabilitate:** Putem folosi direct cod sau biblioteci scrise în Java.



Scala

Scala: Structura Calculatorului

```
1 sealed trait Expr
2 case class Num(v: Double) extends Expr
3 case class BinOp(op: String, l: Expr, r: Expr) extends Expr
```

Cum funcționează cuvintele cheie?

- **sealed trait**: Acționează ca o „umbrelă” care limitează tipurile de date. Programul știe exact că există doar `Num` și `BinOp`.
- **case class**: O clasă specială pentru stocarea datelor. O folosim ca să ne fie ușor să verificăm ulterior ce conține.
- **extends**: Arată că `Num` și `BinOp` fac parte din categoria generală `Expr`.

Scala: Parsare - Identificare Operator

```
1 case "(" :: op :: rest if Set("+", "-", "*", "/").contains(op) =>
2   for {
3     res1 <- parseTokens(rest)
4     // ... continuare procesare operanzi
```

Ce fac operatorii din cod?

- `::`: Taie o listă în bucăți. Extrage prima dată paranteza "(", apoi operatorul op, și păstrează restul.
- `if ... contains()`: Pune o condiție extra: procesează mai departe doar dacă op este un operator valid (+, -, *, /).
- `=>`: Desparte condiția de ceea ce vrem să se întâmple efectiv.

```
1 res1 <- parseTokens(rest)
2 (e1, rest1) = res1
3 res2 <- parseTokens(rest1)
4 (e2, rest2) = res2
5 // Right((BinOp(op, e1, e2), rest_final))
```

Cum extragem datele?

- for { }: Aici îl folosim pentru a lega o serie de operații care ar putea da eroare. Dacă un pas greșește, totul se oprește în siguranță.
- <-: Ia rezultatul dintr-o funcție și ni-l dă spre utilizare.
- (e1, rest1) =: Salvăm expresia găsită în e1 și păstrăm restul simbolurilor în rest1 pentru a căuta următorul element.

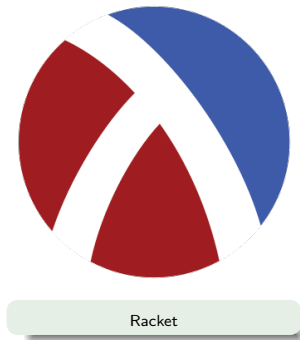
```
1 def eval(e: Expr): Double = e match {  
2   case Num(v) => v  
3   case BinOp("+", l, r) => eval(l) + eval(r)  
4 }
```

Ce face codul?

- `match`: Verifică tipul elementului. E un simplu număr sau e o operație cu două numere?
- `eval(l) + eval(r)`: Funcția se cheamă pe ea însăși ca să calculeze prima dată bucata din stânga (`l`), apoi pe cea din dreapta (`r`), și la final le adună.

2. Racket: Flexibilitate și Liste

- **Bazat pe Liste:** Tot codul se scrie între paranteze și funcționează ca o listă.
- **Extrem de Flexibil:** Nu trebuie să declarăm tipurile variabilelor în avans; programul își dă seama singur din mers.
- **Ușor de Extins:** Ne permite să ne definim propriile reguli gramaticale (macro-uri).




```
1 (define (parseaza expr)
2   (match expr
3     [(? number? n) (num n)]
4     ; ... restul cazurilor
```

Operatorii de potrivire:

- `match`: Seamănă cu cel din Scala; se uită la `expr` și vede ce regulă i se potrivește.
- `?`: Spune programului să aplice un test.
- `number?`: Testul efectiv: „Este elementul ăsta un număr?”. Dacă este, îl asociază cu litera `n`.

```
1 [(list '+ e1 e2) (binop '+ (parseaza e1) (parseaza e2))]  
2 [(list 'sqrt e) (unop 'sqrt (parseaza e))]
```

Cum se citesc listele?

- `list`: Verifică dacă structura arată exact ca o listă cu acele elemente (ex: un operator și două expresii).
- `'` (Apostrof): Transforma textul în simbol. `'+` înseamnă doar cuvântul „plus”, nu calculează nimic încă.
- Funcția se apelează pe sub-elementele `e1` și `e2` ca să descompună complet expresia.

```
1 (define (eval-rkt e)
2   (match e
3     [(num v) v]
4     [(binop '+ l r) (+ (eval-rkt l) (eval-rkt r))]))
```

Detalii de sintaxă:

- []: În Racket, parantezele pătrate fac exact același lucru ca cele rotunde, dar le folosim aici ca să ne fie mai ușor să citim codul.
- (+ ...): Aici simbolul plus nu mai are apostrof! Acesta este apelul real către funcția de adunare din limbaj.

3. Comparație și Performanță

Testul de Viteză

Scala rulează de **1.5 - 2 ori mai repede** decât Racket atunci când are de făcut calcule foarte lungi și repetate.

- **Găsirea erorilor:**

- În Scala, vezi imediat dacă ai greșit când scrii codul (eroare la compilare).
- În Racket, afli dacă ai greșit doar când pornești programul și ajunge la linia respectivă.

4. Lizibilitatea și Concluzii

- **Scala:** Codul este mai sigur și e mai ușor de înțeles de către alți programatori, ideal pentru proiecte mari.
- **Racket:** Codul e mult mai scurt, dar din cauza parantezelor multe, poate deveni o provocare să-l citești.
- **Concluzia noastră:** Scala este mai bună pentru zona de producție/industrie, iar Racket pentru testare rapidă și mediul academic.

 Wikipedia. *Scala (programming language)*.

https:

[`//en.wikipedia.org/wiki/Scala\_\(programming\_language\)`](https://en.wikipedia.org/wiki/Scala_(programming_language))

 Wikipedia. *Racket (programming language)*.

https:

[`//en.wikipedia.org/wiki/Racket\_\(programming\_language\)`](https://en.wikipedia.org/wiki/Racket_(programming_language))

 Google Gemini. *Model de limbaj AI utilizat pentru structurare.*

[`https://gemini.google.com/`](https://gemini.google.com/)

Vă mulțumim!